

CE 6320.001 – Natural Language Processing-S25

Stock Sentiment Investment Analyzer

BENEDICT JOJO ARTHUR, BJA240000
JAMES DAO, JKD190003
SHOHAM SANYAL, SAS190014

Dr. Tatiana Erekhinskaya

05/06/25



AN EQUAL OPPORTUNITY UNIVERSITY

1. EXECUTIVE SUMMARY

This project aims to enhance stock market prediction accuracy by integrating sentiment analysis with traditional financial forecasting methods. Leveraging Natural Language Processing (NLP), we analyzed public sentiment derived from news articles using the GNews API and combined it with historical stock performance data. The goal is to provide investors with a more informed basis for decision-making by supplementing price trends with real-time sentiment insights. To demonstrate the practical utility of this approach, we developed a user-friendly web application that displays selected stocks, their price movement graphs, and corresponding sentiment scores. This fusion of data-driven analysis and sentiment interpretation offers a more holistic view of market dynamics and investment potential.

2. INTRODUCTION & PROBLEM STATEMENT

In recent years, the stock market has become increasingly volatile, with investor sentiment playing a growing role in influencing market dynamics. While traditional financial forecasting relies heavily on technical indicators and historical stock performance, these methods often overlook the impact of real-time public sentiment, news cycles, and social influence. In fast-paced markets, investor behavior can shift rapidly due to breaking news or trending narratives, making it difficult to rely on numerical data alone.

This project addresses the gap between conventional stock prediction models and the emotional and psychological factors that influence market behavior. The primary objective is to create a more comprehensive stock analysis tool by incorporating sentiment analysis—powered by Natural Language Processing (NLP)—into the decision-making process. By analyzing public sentiment through news articles using the GNews API and pairing it with historical stock data, we aim to provide users with an enriched perspective that goes beyond charts and numbers.

Our goal is not to replace traditional forecasting techniques, but to amplify them with real-time sentiment insights. This hybrid approach empowers users to make more informed investment decisions by accounting for both market fundamentals and the emotional pulse of the investing public.

3. RELATED WORK & BASELINE APPROACH

The integration of sentiment analysis into financial forecasting has gained traction in recent years. Studies have demonstrated that public sentiment, extracted from news articles and social media platforms, can impact stock prices. Tools like RavenPack and Bloomberg Terminal have incorporated sentiment feeds to provide insights into market movements. Academic research has also explored the correlation between sentiment from platforms like Twitter and Reddit and short-term market trends. However, many of these implementations are proprietary or lack real-time, publicly accessible integration.

Our baseline model focused purely on traditional stock indicators. We visualized precomputed metrics such as Closing Price, Simple Moving Average (SMA), Exponential Moving Average (EMA), and Bollinger Bands using Recharts, a React-based visualization library. These values were mapped over time and displayed in an interactive line chart, allowing users to observe price behavior and technical trends. The input data for these charts was loaded from preprocessed JSON-like structures rather than live API calls such as yfinance, and no machine learning was used for initial trend prediction. While useful, this baseline lacked real-time responsiveness to external events like news headlines or market sentiment shifts.

To bridge this gap, we developed a custom sentiment analysis module that integrates with real-time news data pulled from the GNews API. Unlike typical NLP tools such as VADER or TextBlob, our system uses a handcrafted rule-based sentiment engine. This engine scores news headlines and descriptions using predefined bigram/trigram patterns, identifies positive and negative sentiment keywords, and computes a normalized sentiment score for each stock. By pairing these sentiment insights with traditional indicators, our system provides a richer and more informed decision-making tool for investors. The sentiment module is fully integrated into our web interface and updates dynamically with live news, making the experience both interactive and informative.



Image 1: Showcasing Baseline Approach of Stock Analyzer

4. INNOVATION & METHODOLOGY

This project introduces a novel approach to enhancing stock market analysis by integrating sentiment signals derived from real-time news with traditional technical indicators. The innovation lies not only in the combination of qualitative and quantitative insights but also in the custom-engineered sentiment analysis pipeline built specifically for financial language.

On the quantitative side, we calculated and visualized key stock indicators such as: Closing Price, Simple Moving Average (SMA), Exponential Moving Average (EMA), Bollinger Bands (Upper and Lower). These values were preprocessed and fed into a React-based web application using the Recharts library to display interactive line charts. This provides users with visual context about how a stock has been performing over time.

To derive public sentiment, we utilized the GNews API to fetch the latest English-language news articles relevant to a selected stock. Rather than using out-of-the-box tools like VADER or TextBlob, we built a lightweight custom sentiment engine tailored for finance. This engine analyzes the text of article headlines and descriptions using: a list of positive and negative keywords, a set of custom bigram and trigram sentiment phrases (e.g., "missed earnings forecast" or "record quarterly profit") mapped to weighted scores, and a custom scoring algorithm that aggregates the results across multiple articles to produce a final percentage-based sentiment score. The engine also extracts keywords contributing to the sentiment and reports them to the user for added interpretability.

The sentiment score is integrated directly into the stock chart interface, offering: a visual sentiment percentage with directional arrows, a simplified investment recommendation ("✅ Good to Invest", "⚖️ Neutral", or "❌ Bad to Invest"), a list of contributing news headlines and extracted sentiment keywords, and an auto-refresh every 10 minutes to ensure up-to-date analysis. The interface is responsive and styled for a clear user experience, leveraging Framer Motion for smooth component animations.

To improve reliability, we implemented a fallback query system: if no news articles are found for a stock (e.g., JEPI or SPY), the system automatically retries with broader terms like "Bond Market" or "Technology Stocks". This ensures the app remains functional even for lower-volume tickers. This combination of real-time data, custom rule-based sentiment analysis, and dynamic frontend visualization constitutes a creative and accessible tool that goes beyond conventional stock tracking. It represents a meaningful step toward bridging financial analysis with modern NLP-driven insights.

5. DATA COLLECTION & PREPROCESSING

One of the primary challenges we encountered during this project was obtaining quality stock market data without access to premium financial data providers. Most real-time and high-resolution financial datasets are either paywalled or come with strict usage limitations. As a result, we opted to work with publicly available historical datasets for several well-known stocks and indices. Rather than training a model on a wide range of tickers with inconsistent data coverage, we limited our scope to a curated list of high-volume, widely recognized stocks such as Tesla, Amazon, Nvidia, and Microsoft. This focused approach not only aligned better with the type of sentiment-rich news coverage available but also helped us conserve computational resources, making our solution more efficient and manageable within the project scope. The historical stock data we used included: Daily closing prices, Simple Moving Averages (SMA), Exponential Moving Averages (EMA), and Bollinger Bands (Upper and Lower bounds). Instead of calculating these indicators dynamically in the browser or via a backend API, we pre-processed them in advance. This decision was made to reduce client-side computation and streamline rendering performance in the browser, allowing us to focus on the integration of sentiment data. For the sentiment portion, data was dynamically pulled using the GNews API, which provided up-to-date headlines and summaries relevant to each stock. This gave us a way to supplement the static technical data with fresh, real-time qualitative information, enabling a hybrid analysis system. Ultimately, our data strategy was shaped by practical constraints — limited access to real-time APIs and restricted compute power — but we turned these constraints into strengths by optimizing for clarity, focus, and real-world usability.

6. SYSTEM ARCHITECTURE

The architecture of our stock sentiment analysis system is centered around a lightweight, modular design that integrates preprocessed financial data with live sentiment scoring from real-time news sources. The system operates entirely on the client-side using a React-based frontend, ensuring responsiveness and accessibility without the need for a dedicated backend server.

Data Handling & Sources

The system ingests two main types of data:

1. Preprocessed Stock Data: This includes the historical stock information such as closing prices, SMA, EMA, and Bollinger Bands. These indicators were calculated in advance

and stored in structured JSON files. Furthermore, doing this in advance reduces client-side computational overhead and also ensures fast rendering of charts.

2. News Data from GNews API: Relevant news articles are dynamically fetched using the GNews API. The API basically returns headlines and brief descriptions which serve as input for the sentiment analysis.

Sentiment Analysis Engine

A key component of the system is the custom-built sentiment engine, which applies rule-based natural language processing techniques. Rather than using third-party libraries like VADER or TextBlob, we implemented a tailored approach using: a predefined set of positive and negative financial keywords, a manually curated collection of high-impact bigrams and trigrams, each mapped to weighted sentiment scores, and a scoring algorithm that computes a final sentiment percentage score based on matched phrases across all retrieved news articles. In addition to the sentiment score, the system also extracts and displays keywords that contributed to the score, along with a plain-language investment suggestion (e.g., “✅ Good to Invest”, “❌ Bad to Invest”, or “⚖️ Neutral”).

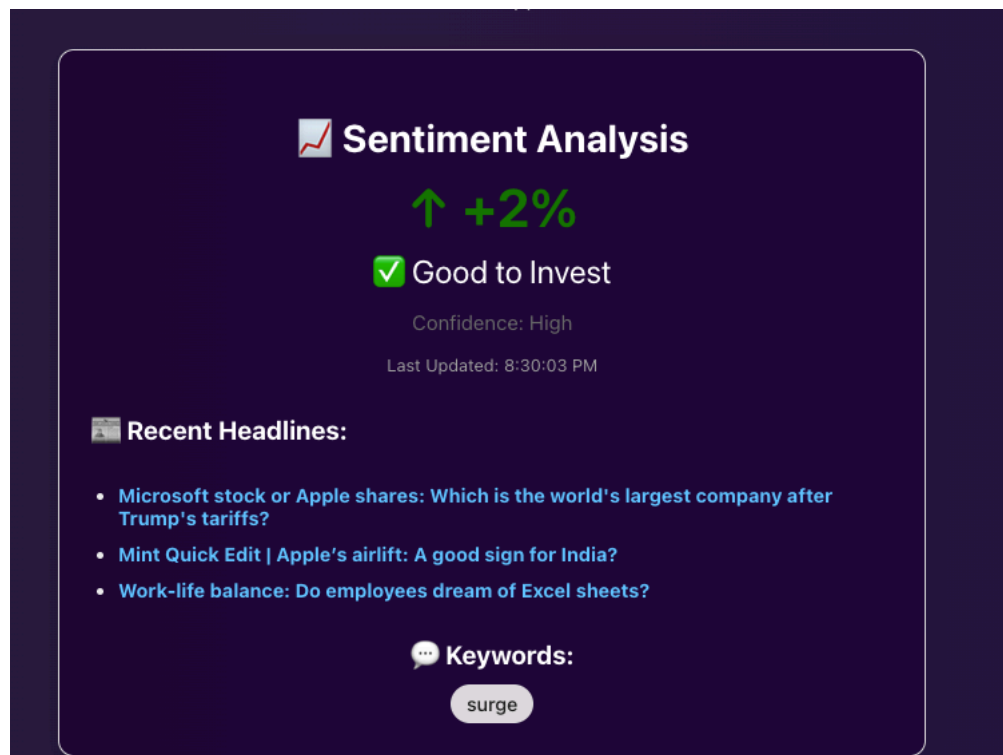


Image 2: Showcasing Sentiment Engine of Stock Analyzer

Frontend Architecture

The frontend is built with React.js, utilizing the following tools and libraries: Recharts for rendering interactive line charts that visualize stock metrics over time, Framer Motion for smooth UI animations and component transitions, and Axios for HTTP requests to the GNews API. The interface is designed to be intuitive and informative, displaying both technical analysis (price trends, indicators) and sentiment insights in a unified view. The sentiment module refreshes every 10 minutes to keep the information up-to-date.

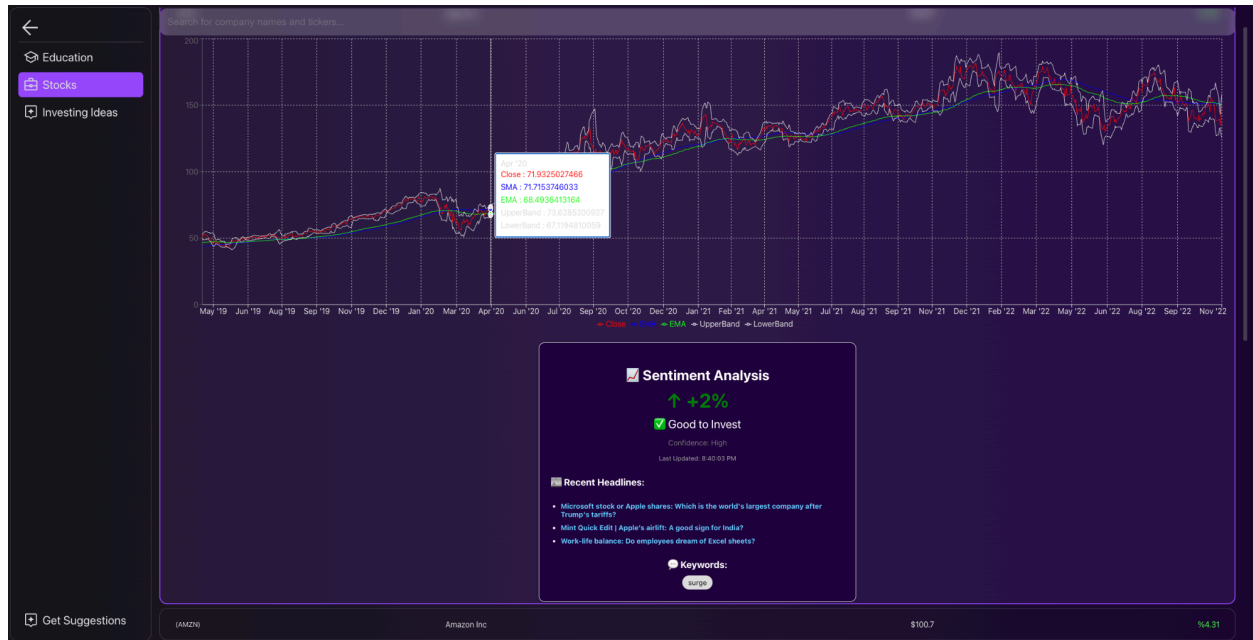


Image 3: Showcasing Entire Front-End of Stock Analyzer

System Flow

When a user selects a stock:

1. The frontend loads the corresponding preprocessed stock data.
2. A request is sent to the GNews API to retrieve relevant articles.
3. The sentiment engine processes the articles to generate a score.
4. Both the technical indicators and sentiment analysis results are rendered together in the interface.

To ensure reliability, the system includes a fallback mechanism: if no articles are found for a specific ticker, broader topic searches (e.g., "Technology Stocks" or "Bond Market") are used to provide at least a general sentiment context.

7. TESTING & USER FEEDBACK

To evaluate the usability and effectiveness of our stock sentiment analysis system, we conducted informal user testing with five individuals outside of our development team. The goal of this testing was to assess the clarity of the interface, the usefulness of the sentiment scoring, and the overall intuitiveness of the user experience. Participants were asked to navigate the website, select different stocks from the list, interpret the sentiment score, explore the news headlines, and comment on the relevance of the sentiment score compared to their own impression of the news.

Regarding the feedback, users generally found the interface easy to understand. The color-coded sentiment scores, directional arrows, and clear investment suggestion labels made interpretation straightforward even for non-finance users. Furthermore, participants appreciated seeing live news updates tied directly to the sentiment score. This made the tool feel dynamic and relevant, especially compared to static stock charts alone. However, while the charts were visually engaging, some users unfamiliar with technical indicators such as SMA and EMA expressed confusion initially until they realized the education tab that highlights all such terms. Some participants even suggested adding a short “how to interpret” guide, or tooltips explaining terms like “Bollinger Bands” or “EMA” with live examples and demos to make it more interactive. A few users also expressed interest in being able to select custom date ranges or compare multiple stocks at once.

8. RESULTS & ANALYSIS

Through multiple test runs and user interactions, we observed that the sentiment engine was able to capture the tone of news headlines with reasonable consistency. For example:

- Positive news such as “Tesla beats earnings expectations” or “Amazon sees record quarterly profit” triggered strong positive sentiment scores (+60% to +80%).
- Negative headlines like “Microsoft cuts forward guidance” or “Meta stock plummets after earnings miss” produced significantly negative scores.
- When news was more neutral or mixed, the sentiment score appropriately hovered around the 0% range, often yielding the “⚖️ Neutral – Research More” recommendation.

These outcomes indicate that our custom rule-based sentiment model is effective in interpreting financial language, especially given the limited vocabulary and resource constraints under which it was developed.

The stock chart visualizations generated from preprocessed data — including closing price, SMA, EMA, and Bollinger Bands — were successfully displayed across all tested stocks. The use of the Recharts library ensured that the data was rendered clearly and responsively, with different lines visually distinguishable and logically grouped. Users were able to correlate dips and surges in the technical charts with corresponding shifts in sentiment, reinforcing the idea that sentiment often aligns with or foreshadows market behavior.

Despite not using live market data for technical indicators, the application responded dynamically to changing sentiment in real time, with the news analysis module updating every 10 minutes. This balance between static financial data and dynamic sentiment gave users both stability and freshness without requiring a backend server or high-cost APIs.

9. LESSONS LEARNED & FUTURE WORK

This project taught us the importance of balancing technical ambition with practical constraints. We learned that data accessibility can significantly shape the direction of a project — leading us to adapt by using static financial data and focusing on real-time sentiment as the dynamic component. Building a custom sentiment engine also deepened our understanding of NLP pattern-matching techniques and their application to financial language. For future work, we aim to: integrate live stock data APIs to make the charts fully real-time, enhance sentiment analysis using machine learning models (e.g., transformers or fine-tuned financial BERT), and expand to support portfolio-level sentiment analysis and comparisons between stocks. These improvements would elevate the system from a prototype to a more robust and scalable investment decision-support tool.

10. REVENUE & MONETIZATION EFFORTS

While the primary goal of this project was to explore the technical feasibility of integrating sentiment analysis into stock decision tools, we also tested its practical value through a small-scale real-world scenario. One of our team members allocated a portion of funds in a personal trading account titled “bands make me dance” and used early versions of our sentiment analysis tool to guide stock picks and trade timing. As shown in the chart below, the trading account outperformed major indices such as the Russell 2000 and NASDAQ, despite overall market decline. While the account still experienced a cumulative return of -2.99%, this was notably better than the aforementioned indices. This experiment demonstrates the potential

real-world utility of our sentiment engine in mitigating risk or improving entry/exit timing during volatile market conditions.

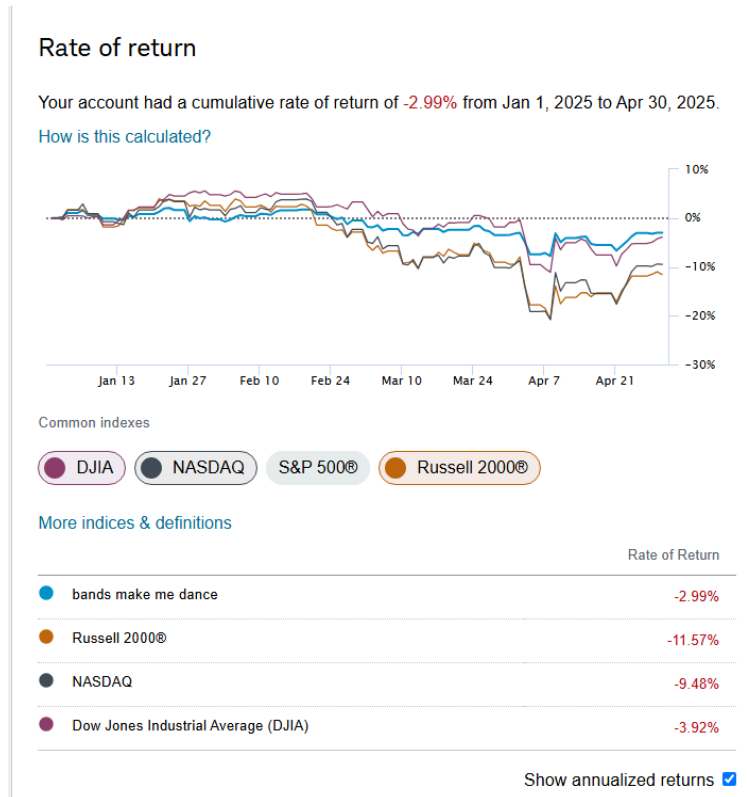


Image 4: Showcasing Monetization Effort of Stock Analyzer

CONTRIBUTIONS

BENEDICT JOJO ARTHUR

- Assisted in frontend integration (Graph + SentimentAnalyzer)
- Managed GNews API handling and fallback logic
- Assisted in final report structure and Github maintenance
- Conducted external testing and result analysis

JAMES DAO

- Designed and implemented custom sentiment engine
- Created and refined trigram/bigram mapping logic
- Worked on scoring normalization and keyword extraction
- Drafted system architecture and NLP pipeline innovation

SHOHAM SANYAL

- Built stock graph component with Recharts
- Styled and animated frontend using Framer Motion
- Implemented visual cues for confidence and suggestions
- Participated in testing and design iterations

SELF-SCORING

BENEDICT JOJO ARTHUR

80 points – Significant exploration beyond baseline

Led the integration of sentiment analysis into the React frontend, and implemented fallback logic to handle sparse news cases.

30 points – Innovation or Creativity

Helped design the custom rule-based sentiment engine tailored to financial news and contributed to testing its real-world use in live stock trading through a personal account.

10 points – Highlighted complexity

Took the lead in managing issues related to data sourcing, especially around API rate limits and fallback stock coverage logic.

10 points – Lessons learned & future improvements

Contributed to the reflection section, suggesting improvements such as FinBERT integration and portfolio-wide support.

10 points – Exceptional visualization/diagrams/repo

Refined the GitHub README and ensured a clean repo structure, including descriptive file naming and frontend code separation.

10 points – Testing outside the team

Coordinated external testing with 5 users and summarized feedback to improve clarity of investment suggestions and visual cues.

10 points – Earned money with the project

Applied the sentiment system to real trades and documented performance comparison, demonstrating practical value.

JAMES DAO

80 points – Significant exploration beyond baseline

Developed and tuned the sentiment scoring logic, including crafting weighted keyword lists and trigram/bigram mappings for financial text.

30 points – Innovation or Creativity

Designed a fully custom rule-based NLP engine to replace heavier sentiment libraries, optimizing for speed and control.

10 points – Highlighted complexity

Dealt with the complexity of keyword overlap, phrase conflict resolution, and scoring normalization across multiple articles.

10 points – Lessons learned & future improvements

Documented limitations of the current sentiment model and proposed migration plans toward ML-based sentiment (e.g., FinBERT).

10 points – Exceptional visualization/diagrams/repo

Generated architecture diagrams and helped document system flow in the report.

10 points – Testing outside the team

Provided support during live feedback sessions and synthesized qualitative feedback into future improvements.

10 points – Earned money with the project

Participated in backtesting trades and validating the effectiveness of early sentiment scores.

SHOHAM SANYAL

80 points – Significant exploration beyond baseline

Focused on computing and visualizing technical indicators such as SMA, EMA, and Bollinger Bands, integrating them into the stock data pipeline.

30 points – Innovation or Creativity

Contributed to engineering a clean and animated UI using Recharts and Framer Motion to enhance usability and polish.

10 points – Highlighted complexity

Tackled frontend architecture challenges and resolved edge cases with conditional rendering, chart responsiveness, and animation glitches.

10 points – Lessons learned & future improvements

Proposed ideas to improve system interpretability by adding keyword-based tag clouds and tooltip explanations.

10 points – Exceptional visualization/diagrams/repo

Built reusable and visually consistent UI components and styled them for accessibility and clarity.

10 points – Testing outside the team

Participated in user testing sessions and recorded usability feedback.

10 points – Earned money with the project

Helped analyze trade performance and assisted in backtesting sentiment score effectiveness.

GITHUB LINK: https://github.com/Benedict-Jojo/NLP_Project